Warsaw Tram Delay Prediction Platform

Architecture Report

Authors

Wiktoria Koniecko
Michał Szewczak
Sebastian Trojan

May 8, 2026

Contents

System Overview

The platform ingests three categories of data from the Warsaw city API (`api.um.warszawa.pl`):

- **Timetable data** (static, loaded once daily before 04:00) — stop locations, line-stop assignments, and per-brigade precise schedules for every tram stop–line–brigade combination in the network.
- **Vehicle position data** (streaming, 04:00–01:00) — live GPS pings for every active tram: `Lat`, `Lon`, `Time`, `Lines`, `Brigade`, `VehicleNumber`.
- **Weather data** (periodic, throughout the day) — temperature, precipitation, wind, and visibility for Warsaw.

The platform derives tram delays by matching live positions against the loaded timetable, stores both raw and feature data in BigQuery, trains a delay prediction model on Vertex AI, and serves two user-facing interfaces: a real-time tram map with computed delays and a prediction UI.

The system scope is **trams only**. Operating window is **04:00–01:00 daily** (21 hours). Outside this window no position data arrives and all streaming components are idle.

Timetable Loading Procedure

The timetable loader runs as a Cloud Run Job triggered by Cloud Scheduler at **03:00 daily**, completing before tram service begins at 04:00. The loading procedure is sequential and hierarchical:

1. **Download all stops.** `GET /api/action/dbstore_get` with the stops resource ID returns the full list of Warsaw tram stops: stop ID, stop number, stop name, lat, lon, direction.
2. **For each stop: download all lines.** `GET /api/action/dbtimetable_get` parameterised by stop ID returns the list of tram lines passing through that stop.
3. **For each stop–line pair: download brigade schedules.** A further call to `dbtimetable_get` parameterised by stop ID and line number returns all brigades serving that combination, together with precise scheduled departure times throughout the day.

The result is a complete timetable: for every (stop, line, brigade) triple the system knows the scheduled departure time at every point in the day. This is the reference used by the Stream Processor to compute `delay_seconds` during operating hours.

Volume estimate. Warsaw has approximately 300 tram stops, 30 tram lines, and on average 5–8 brigades per line per stop. The loader makes roughly $300 + 300 \times 30 + 300 \times 30 \times 6 \approx 63\,000$ sequential API calls. At a conservative 200 ms per call this takes ≈3.5 hours, which is too slow. The loader must therefore parallelise the stop–line and stop–line–brigade calls using a bounded async pool (20–50 concurrent requests), reducing wall-clock time to under 30 minutes.

Microservice Map

Service inventory

Table 1: Microservice inventory

Service	Runtime	Responsibility	Trigger / schedule
Timetable Loader	Cloud Run Job	Sequential–parallel download of stops, lines, brigades; write to BigQuery	Cloud Scheduler 03:00 daily
Position Ingestor	Cloud Run Job	Poll tram positions every 60 s; publish to Pub/Sub	Cloud Scheduler every 60 s, 04:00–01:00
Weather Ingestor	Cloud Run Job	Poll weather API; publish to Pub/Sub	Cloud Scheduler every 10 min
Stream Processor	Dataflow (streaming)	Consume Pub/Sub; match position to timetable; compute delay; write BQ + Firestore	Pub/Sub push; auto-scales; idle outside traffic hours
Feature Builder	BQ scheduled query	Aggregate rolling features for model training	BigQuery scheduled query, 02:00 daily
Training Job	Vertex AI Custom Job	Read BQ features; train model; write artefact to GCS	Weekly or on-demand
Prediction API	Cloud Run + Vertex AI Endpoint	Serve delay predictions via REST	HTTP request; scales to 0

Connection diagram

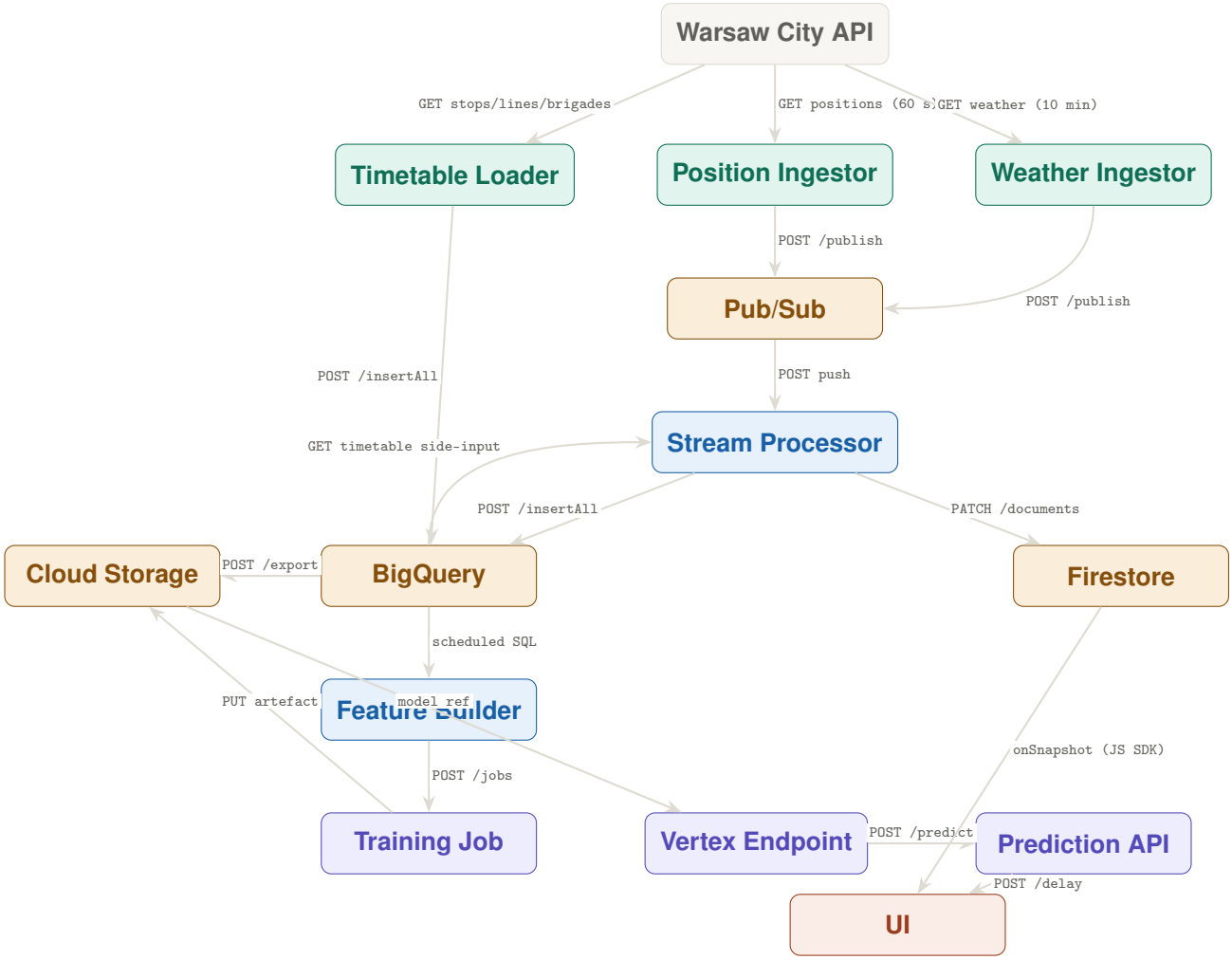

Figure 1: Microservice connection diagram. Teal = ingestion; blue = processing; purple = ML serving; orange = storage; coral = frontend. The timetable BigQuery tables act as a shared side-input: loaded once by the Timetable Loader, read continuously by the Stream Processor. The UI reads live vehicle state directly from Firestore via the Firestore JS SDK `onSnapshot` listener.

API Design

Protocol selection principles

The default protocol for all boundaries is **REST/HTTPS with JSON payloads**. A non-REST protocol is used only where a concrete, quantified benefit justifies the added operational complexity. Two cases qualify in this system; both are identified below.

External API calls (Warsaw city API)

All Warsaw API endpoints are polled over plain HTTP GET and return JSON. No authentication beyond an API key in the query string is required.

Table 2: External REST calls

Caller	Method	Endpoint (abbreviated)	Cadence
Timetable Loader	GET	`/dbstore_get` (stops)	Once at 03:00
Timetable Loader	GET	`/dbtimetable_get` (lines per stop)	Once at 03:00
Timetable Loader	GET	`/dbtimetable_ge` (brigades per stop–line)	Once at 03:00
Position Ingestor	GET	`/wsstore_get` (tram positions)	Every 60 s
Weather Ingestor	GET	`/imgw_get` (Warsaw weather)	Every 10 min

Internal service APIs

Table 3: Internal API contracts

From → To	Protocol	Operation	Justification
Timetable Loader → BigQuery	REST	`POST /insertAll`	Standard BQ streaming insert
Position Ingestor → Pub/Sub	REST	`POST /{topic}:publish`	Batch up to 1 000 msgs
Weather Ingestor → Pub/Sub	REST	`POST /{topic}:publish`	Same topic, different `source` attr
Pub/Sub → Stream Processor	**Pub/Sub push***	POST to Cloud Run URL	Managed delivery; GCP handles retries
Stream Processor → BigQuery	REST	`POST /insertAll`	Micro-batched rows
Stream Processor → Firestore	REST	`PATCH /{document}`	Upsert vehicle state
Stream Processor ← BigQuery (side input)	**gRPC†**	BQ Storage Read API	See note below
Frontend → Firestore	Firestore JS SDK	`onSnapshot` listener	Direct browser access; push updates; no intermediate service
Frontend → Prediction API	REST	`POST /v1/delay`	Request–response

***Pub/Sub push vs. pull.** Push delivery (GCP POSTs to the Stream Processor's Cloud Run URL) is used rather than pull because it integrates natively with Cloud Run's concurrency model and eliminates long-polling overhead. This is not a separate protocol — it is still HTTP — but it inverts the connection direction.

†**gRPC for BigQuery Storage Read API.** The Stream Processor loads the daily timetable as a Dataflow side input. The BigQuery Storage Read API uses gRPC + Apache Arrow over the wire and achieves **10–40× higher throughput** than the REST JSON `tabledata.list` endpoint for bulk reads (Google benchmark: ~10 GB/s vs. ~300 MB/s). At daily timetable size (≈50–200 MB) the difference is modest, but this API is the standard Dataflow BigQuery source connector and requires no additional code or schema management on our side. It is the only gRPC surface in the system and is encapsulated inside the Dataflow runner — no custom Protobuf schemas are maintained by the team.

Prediction API — endpoint specification

Base URL: `https://api.trams.warsaw/v1`

Table 4: Prediction API endpoints

Method	Path	Auth	Description
POST	`/delay`	API key	Predict delay for a tram at a given stop and time
GET	`/health`	None	Liveness probe

POST /delay — request:

```
{
  "line":      "3",
  "stop_id":   "1001",
  "brigade":   "1",
  "timestamp": "2025-09-01T08:15:00Z"
}
```

POST /delay — response:

```
{
  "delay_seconds":  134,
  "confidence":     0.78,
  "model_version":  "v2.1.0",
  "features_used":  ["precip_mm", "route_avg_delay_30m",
                     "hour_of_day", "brigade_delay_history"]
}
```

Firestore live vehicle data

The Frontend SPA reads live vehicle state directly from Firestore using the Firestore JS SDK `onSnapshot` listener, receiving push updates as documents change. Each document in the `vehicles` collection is keyed by `vehicle_number` and has the following shape:

```
{
  "vehicle_number": "3456",
  "line":           "3",
  "brigade":        "1",
  "lat":            52.2297,
  "lon":            21.0122,
  "delay_s":        134,
  "next_stop_id":   "1002",
  "updated":        1725174897
}
```

The `updated` timestamp allows the frontend to expire vehicles not seen for more than 120 s. Firestore security rules restrict reads to authenticated users; no service credentials are embedded in the client bundle.

Staleness budget. Warsaw API publishes positions every 60 s. The Stream Processor adds up to 90 s of processing latency (p95). Firestore push delivery adds negligible latency ($<$1 s). Total end-to-end staleness: $\approx$151 s, dominated by the 60 s source cadence — not by any polling interval.

Storage Characteristics

Table 5: Storage characteristics per data store

Store	Structure	SQL / NoSQL	Consistency	Access	Est. volume	Notes
BQ timetable	Structured	SQL	Strong[α]	R & W	50–200 MB/day	Written once at 03:00; read-only during traffic hours
BQ positions (raw)	Structured	SQL	Eventual	R & W	3–8 GB/month	Append-only; partitioned by `date`
BQ positions (enriched)	Structured	SQL	Eventual	R & W	5–15 GB/month	Includes `delay_s`, next stop, weather join; training source
BQ features	Structured	SQL	Eventual	R & W	1–3 GB/month	Daily rolling aggregates; clustered on `line`, `hour`
Cloud Storage	Unstructure	—	Strong (obj)	R & W	50–200 MB/model	Raw JSON archive + model artefacts
Firestore	Semi-struct.	NoSQL (doc)	Strong (per-doc)	R & W	$<$100 MB	Live vehicle state; TTL 2 h; read directly by frontend via JS SDK
Pub/Sub	Unstructure	—	At-least-once	W / R	$<$500 MB/month	7-day retention; not a durable store

[a] The timetable table is written once and is fully visible before the streaming pipeline reads it. BigQuery eventual consistency is not a concern here because the Timetable Loader will be set to finish before the start of other reads.

BigQuery dataset structure

BigQuery holds four logical tables, all in the same dataset `trams_warsaw`:

`timetable`
: Written daily at 03:00 by the Timetable Loader. Schema: `stop_id`, `stop_name`, `lat`, `lon`, `line`, `brigade`, `scheduled_departure` (TIMESTAMP). Partitioned by `load_date`. **Read-only during traffic hours** — the Stream Processor loads it as a Dataflow side input once at pipeline startup.

`positions_raw`
: Append-only. One row per GPS ping from the Warsaw API. Fields: `vehicle_number`, `line`, `brigade`, `lat`, `lon`, `gps_time`, `ingested_at`. Partitioned by `DATE(ingested_at)`.

`positions_enriched`
: Computed by the Stream Processor. Extends `positions_raw` with: `matched_stop_id`, `scheduled_departure`, `delay_s`, `precip_mm`, `temp_c`. Partitioned by `DATE(gps_time)`, clustered on `line`. This is the primary training input.

`features`
: Materialised daily by the Feature Builder (BigQuery scheduled SQL). Rows represent (line, brigade, stop, hour-of-day) aggregates over a rolling 90-day window: mean delay, delay standard deviation, rain–delay correlation, peak-hour flag. Clustered on `line`, `hour_of_day`.

Firestore

Firestore stores only the **current operational state**: one document per active tram, keyed by `vehicle_number`. Documents are overwritten (upserted) by the Stream Processor on every position update and expire via TTL after 2 hours of inactivity — preventing accumulation of stale entries for trams that have completed service.

The Frontend SPA subscribes to the `vehicles` collection via the Firestore JS SDK `onSnapshot` listener, receiving push updates as documents change. Firestore security rules restrict reads to authenticated users only.

Outside traffic hours (01:00–04:00) no writes occur and the collection drains to empty via TTL.

Cloud Storage

Two buckets:

- `trams-raw-archive` — daily JSON exports from `positions_raw`; 60-day lifecycle rule then auto-delete.
- `trams-ml-artefacts` — versioned model files written by the Training Job; never auto-deleted; referenced by the Vertex AI Endpoint.

SLA, SLO and SLI

All availability SLOs are measured **within the 04:00–01:00 traffic window only**. Services that are legitimately idle outside this window are not penalised for being stopped.

Per-service SLI / SLO / SLA

Table 6: SLI, SLO and SLA per service

Service	SLI	SLO	SLA	Window
Timetable Loader	Load completion before 04:00	99%	95%	Daily (30-day rolling)
	Full timetable row count vs. prior day $\pm$10%	99%	95%	Per run
Position Ingestor	Successful poll ratio (Warsaw API non-5xx)	95%	90%	Rolling 24 h
	Position freshness: age of newest ping in Pub/Sub	$\leq$90 s	$\leq$3 min	p99 / 1 h
Stream Processor	Message processing latency (push $\rightarrow$ BQ write)	$\leq$90 s	$\leq$3 min	p95 / 1 h
	Pipeline availability (traffic hours)	99.5%	99%	Rolling 30 d
Prediction API	Request success rate (HTTP 2xx)	99.5%	99%	Rolling 7 d
	Prediction latency (end-to-end)	$\leq$800 ms	$\leq$2 s	p95 / h
Training Job	Age of deployed model	$\leq$7 d	$\leq$14 d	Continuous
Frontend SPA	Page load time (LCP)	$\leq$2.5 s	$\leq$4 s	p75 / 24 h
	Availability (Firebase Hosting)	99.9%	99.5%	Rolling 30 d

Error budgets (30-day traffic-hours window)

Table 7: Monthly error budgets

Service / SLI	SLO	Budget (min/month)	Notes
Stream Processor availability	99.5%	189 min	$\approx$3.1 h/month at 21 h/day
Prediction API success rate	99.5%	189 min	Vertex cold-start main risk
Frontend availability	99.9%	38 min	Firebase SLA covers most of this

Budgets are computed over the 21-hour traffic window only ($30 \times 21 \times 60 = 37\,800$ minutes/month), not the full calendar month.

Timetable Loader risk

The Timetable Loader must complete within the 03:00–04:00 window (60 min). If it overruns or fails, the Stream Processor starts with a stale timetable (yesterday's), which is acceptable for

most days but will produce incorrect delays on days when the schedule changes significantly (e.g. first day after a timetable revision). Mitigations:

1. Loader writes a `load_status` document to Firestore on completion; the Stream Processor checks this at startup and emits a Cloud Monitoring alert if the timetable is more than 26 hours old.
2. Bounded async concurrency (40 workers) keeps wall-clock load time under 20 minutes, leaving 40 minutes of margin.
3. On failure, Cloud Run Job retries automatically (max 3 attempts).

Vertex AI cold-start risk

With `min-replica-count: 0`, a Vertex Endpoint cold start takes 25–45 s, breaching the p95 800 ms SLO. Mitigation options in order of cost:

1. `min-replica-count: 1` during 04:00–01:00 via Cloud Scheduler toggle — adds $15–25/month.
2. Cache common (line, stop, hour) predictions in Cloud Memorystore (Redis) — eliminates most cold starts at $10/month.
3. Accept the SLO breach and commit the SLA to p95 $\leq$2 s, which cold starts satisfy.

Summary

Table 8: Architecture decision summary

Dimension	Decision
Scope	Trams only; 04:00–01:00 operating window
Data sources	Warsaw city API only (timetable, positions, weather)
Timetable ingestion	3-level sequential–parallel hierarchy (stops $\rightarrow$ lines $\rightarrow$ brigades); written to BQ before 04:00 daily
Delay computation	Computed on our side by matching GPS positions to the loaded timetable; no ZTM-side delay feed exists
API protocols	REST/HTTPS default; gRPC used only for BQ Storage Read API (Dataflow built-in, no team-owned schema); Firestore JS SDK for frontend live data
Primary storage	BigQuery (timetable + enriched positions + features) + Firestore (live vehicle state, read directly by frontend via JS SDK)
ML platform	Vertex AI Training + Vertex AI Endpoint; artefacts in GCS
Cost strategy	Scale-to-zero on all Cloud Run services; partitioned/clustered BQ; no always-on GPU